📝 퀴즈 1 — Day 1~5 복습 (50문항, 4지선다)

> `중간점검_Day1-5복습_Day6-14미리보기.md` 기반. 먼저 안 보고 풀고 → 맨 뒤 **[답지와 해설]**로 채점 → 틀린 것만 본문 다시 보기. 완료 기준: 점수보다 "왜 그 답인지 말로 설명 가능한지".

---

1. 손님 요청이 지나가는 올바른 순서는?

- ① router → main → models → schemas → database
- ② main → router → database → models → schemas → 응답
- ③ models → schemas → main → router → 응답
- ④ database → main → router → schemas → models

2. `main.py`의 역할을 가장 잘 말한 것은?

- ① DB 테이블 모양을 정의한다
- ② 앱을 켜고, 요청을 어느 라우터로 보낼지 연결한다
- ③ 주고받는 데이터를 검사한다
- ④ AI에게 프롬프트를 보낸다

3. `routers/*.py` 파일이 하는 일은?

- ① DB 연결을 열고 닫는다
- ② 앱을 실행시킨다
- ③ 요청을 받아 실제로 처리한다(넣기/꺼내기 로직)
- ④ 테이블 설계도를 그린다

4. `models.py`를 식당에 비유하면?

- ① 안내데스크
- ② 주문서 양식
- ③ 창고 선반 설계도
- ④ 창고 문지기

5. `schemas.py`의 역할을 비유하면?

- ① 창고 선반 설계도
- ② 주문서/영수증 양식 (데이터 검사)
- ③ 주방 코너
- ④ 회사 대표

6. `database.py`가 담당하는 일은?

- ① 라우터 연결
- ② DB 연결을 열고 닫기(get_db)
- ③ 프롬프트 조립
- ④ 응답 JSON 모양 정리

7. 파일을 여러 개로 나누는 진짜 이유는?

- ① 코드가 어려워서 어쩔 수 없이
- ② 역할별로 나눠야 관리가 쉬워서
- ③ 파일이 많아야 빨라서
- ④ FastAPI가 강제해서

8. 새 기능을 만들 때 반복되는 "4단계 패턴"의 순서는?

- ① schemas → models → main → router
- ② models → schemas → router → main
- ③ router → main → models → schemas
- ④ main → models → router → schemas

9. Day 2에서 다룬 "대상"은?

- ① 수업기록(LessonRecord)
- ② 피드백(Feedback)
- ③ 아이(Child)
- ④ 프롬프트

10. Day 3에서 다룬 "대상"은?

- ① 아이(Child)
- ② 수업기록(LessonRecord)
- ③ 피드백(Feedback)
- ④ 벡터

11. Day 5에서 다룬 "대상"은?

- ① 아이(Child)
- ② 수업기록(LessonRecord)
- ③ 피드백(Feedback)
- ④ RAG 문서

12. POST와 PATCH의 차이로 맞는 것은?

- ① POST=수정, PATCH=생성
- ② POST=새 데이터 생성, PATCH=기존 데이터 일부 수정
- ③ 둘 다 삭제
- ④ POST=조회, PATCH=조회

13. `child.model_dump(exclude_unset=True)`가 하는 일은?

- ① 모든 필드를 None 포함해서 다 꺼낸다
- ② 사용자가 실제로 보낸 필드만 꺼낸다
- ③ DB를 삭제한다
- ④ 문자열을 딕셔너리로 바꾼다

14. `setattr(db_child, "age", 9)`와 같은 뜻은?

- ① `db_child.age == 9`
- ② `db_child.age = 9`

- ③ `getattr(db_child, "age")`
- ④ `del db_child.age`

15. 두 테이블에 `relationship`을 양쪽 다 써야 하는 이유는?

- ① 안 그러면 서버 켤 때 에러가 나서
- ② 속도가 빨라져서
- ③ DB 용량을 줄이려고
- ④ 안 써도 상관없다

16. `GET /children`(전체)과 `GET /children/{id}`(하나)의 코드 끝이 각각 맞는 것은?

- ① 둘 다 `.all()`
- ② 전체 `.first()` / 하나 `.all()`
- ③ 전체 `.all()` / 하나 `.first()`
- ④ 둘 다 `.first()`

17. `.filter(LessonRecord.child_id == child_id)`에서 왼쪽/오른쪽은?

- ① 왼쪽=들어온 값, 오른쪽=DB 컬럼
- ② 왼쪽=DB 컬럼, 오른쪽=URL로 들어온 값
- ③ 둘 다 DB 컬럼
- ④ 둘 다 URL 값

18. `id`(기본키)를 가장 잘 설명한 것은?

- ① 다른 테이블을 가리키는 끈
- ② 그 표에서 자기 자신의 고유 번호
- ③ 저장 시각
- ④ 아이 이름

19. `child_id`(외래키)의 뜻은?

- ① 이 기록이 어느 아이 것인지 가리키려고 children의 id를 베껴 적은 칸
- ② 수업 날짜
- ③ 기록의 자기 고유번호
- ④ 강사 메모

20. 저장(POST) 3종 세트 순서는?

- ① commit → add → refresh
- ② add → commit → refresh
- ③ refresh → add → commit
- ④ add → refresh → commit

21. POST에서 `created_at`을 파라미터로 받지 않는 이유는?

- ① 필요 없는 값이라서
- ② models.py의 `server_default=func.now()`로 DB가 자동 입력해서
- ③ 사용자가 몰라서
- ④ 에러가 나서

22. Pydantic v2에서 `orm_mode = True`는 무엇으로 바뀌었나?

- ① `from_orm = True`
- ② `from_attributes = True`
- ③ `orm = True`
- ④ `attributes_mode = True`

23. 모델에 새 컬럼을 추가했는데 DB에 반영이 안 됐을 때, 연습 데이터라서 쓴 방법은?

- ① 서버를 껐다 켠다
- ② `drop_all()`로 지우고 `create_all()`로 다시 생성
- ③ 컬럼을 지운다
- ④ 아무것도 안 해도 자동 반영된다

24. `prompt_service.py`의 역할은?

- ① Gemini에 실제로 요청을 보낸다
- ② 아이 정보+기록을 글(프롬프트)로 조립한다 (아직 AI 안 부름)
- ③ DB에 저장한다
- ④ 화면을 그린다

25. `llm_service.py`의 역할은?

- ① 프롬프트를 조립만 한다
- ② 조립된 글을 실제로 Gemini에 보내고 답을 받아온다
- ③ 테이블을 만든다
- ④ 라우터를 연결한다

26. f-string에 대한 설명으로 맞는 것은?

- ① 문자열 앞에 `f`를 붙이고 `{변수}` 자리에 값이 들어간다
- ② 파일을 여는 함수다
- ③ 반복문이다
- ④ DB 조회 메서드다

27. `load_dotenv`와 `client`를 함수 "밖"에 두는 이유는?

- ① 함수 안에 못 넣어서
- ② 호출할 때마다 .env를 다시 읽거나 client를 또 만드는 낭비를 막으려고
- ③ 보안 때문에
- ④ 문법 오류라서

28. API 키를 다루는 올바른 방법은?

- ① 코드에 직접 적는다
- ② GitHub에 올린다
- ③ `.env`에 숨기고 `.gitignore`로 제외한다
- ④ 주석으로 남긴다

29. `json.loads(result)`가 하는 일은?

- ① 딕셔너리를 문자열로 바꾼다

- ② 문자열을 파이썬 딕셔너리로 바꾼다
- ③ 파일을 삭제한다
- ④ DB에 저장한다

30. `HTTPException(status_code=404)`를 일부러 발생시키는 이유는?

- ① 서버를 끄려고
- ② 없는 ID로 크래시(500) 나기 전에 "없음"을 깔끔히 알려주려고
- ③ 속도를 높이려고
- ④ 로그를 남기려고

31. 여러 개(리스트)를 돌려줄 때 응답 모델을 어떻게 감싸나?

- ① `response_model=FeedbackResponse`
- ② `response_model=List[FeedbackResponse]`
- ③ `response_model=dict`
- ④ 감쌀 필요 없다

32. 스키마에 `from_attributes = True`가 필요한 이유는?

- ① 속도 때문에
- ② DB 객체(.점으로 읽는)를 Pydantic이 읽어서 JSON으로 만들게 허락하려고
- ③ 보안 때문에
- ④ 없어도 된다

33. "매개변수(parameter)"를 가장 잘 비유한 것은?

- ① 이미 정해진 고정값
- ② 부를 때 채워질 "빈 그릇"
- ③ DB 테이블
- ④ 응답 코드

34. Swagger에서 "진짜 결과"는 어디를 봐야 하나?

- ① 아래쪽 "Responses" 표
- ② 위쪽 "Server response"의 Code
- ③ 브라우저 주소창
- ④ 터미널 색깔

35. 라우터를 만들고 나서 main.py에 반드시 써야 하는 2줄은?

- ① print + return
- ② import + include_router
- ③ add + commit
- ④ filter + limit

36. 요청 흐름을 말로 외우는 버전으로 맞는 것은?

- ① 응답 → schemas → models → database → router → main
- ② main(어디로) → router(처리) → database(연결) → models(DB) → schemas(모양) → 응답
- ③ router → schemas → main → 응답

- ④ models → main → schemas → 응답

37. `.all()`과 `.first()`의 차이는?

- ① `.all()`=하나 / `.first()`=여러 개
- ② `.all()`=여러 개(리스트) / `.first()`=딱 하나
- ③ 둘 다 같다
- ④ `.all()`=삭제 / `.first()`=수정

38. `Create` 스키마와 `Response` 스키마의 차이는?

- ① Create=줄 때 / Response=받을 때
- ② Create=받을 때 / Response=줄 때(보통 id·created_at 추가)
- ③ 완전히 같다
- ④ Response엔 아무것도 없다

39. 안 쓰는 변수 `rag_context`는 어느 Day에서 채우기로 예약된 것?

- ① Day 2
- ② Day 5
- ③ Day 8 (RAG)
- ④ Day 14

40. f-string 안에서 글자 그대로 중괄호 `{`를 쓰려면?

- ① `{` 하나
- ② `{{` 두 개
- ③ `\{`
- ④ `((`

41. 문자열에서 `\n`의 뜻은?

- ① 공백
- ② 줄바꿈(엔터)
- ③ 탭
- ④ 따옴표

42. `records_text += "..."`에서 `+=`의 뜻은?

- ① 값을 비교한다
- ② 기존 문자열 뒤에 더 붙인다(누적)
- ③ 문자열을 지운다
- ④ 숫자를 곱한다

43. 파라미터 이름을 클래스 이름과 똑같이(`LessonRecord`) 지으면 생기는 문제는?

- ① 아무 문제 없다
- ② 클래스인지 파라미터인지 Python이 헷갈린다
- ③ 속도가 느려진다
- ④ DB가 지워진다

44. `data["next_goal"]`에서 key 이름을 틀리게 쓰면?

- ① 자동으로 맞춰준다
- ② `KeyError`가 난다
- ③ None을 준다
- ④ 서버가 켜진다

45. 이 프로젝트의 AI(LLM)는 무엇을 쓰나?

- ① OpenAI GPT
- ② Google Gemini
- ③ Llama
- ④ Claude

46. 키 인증까지 됐는데 `429 RESOURCE_EXHAUSTED`가 나면 원인은?

- ① 코드 문법 오류
- ② 코드 문제가 아니라 할당량/모델 문제
- ③ 인터넷 끊김
- ④ DB 오류

47. `llm_service.py` 돌리기 전에 확인할 것은?

- ① DB 비우기
- ② venv(가상환경)가 켜졌는지 `(kids-feedback)` 표시 확인
- ③ 브라우저 새로고침
- ④ 파일 이름 바꾸기

48. Swagger가 무엇인지 맞는 것은?

- ① 진짜 사용자용 웹페이지
- ② 자동 생성되는 API 테스트 화면(개발자용)
- ③ 데이터베이스
- ④ AI 모델

49. 복습의 "완료 기준"은?

- ① 노트를 많이 쓴 것
- ② 내 말로 30초 설명 가능한지
- ③ 문서를 다 읽은 것
- ④ 시간이 지난 것

50. 이 프로젝트를 한 문장으로?

- ① 게임을 만드는 서버
- ② 강사가 아이 정보·기록을 넣고 꺼내보고, AI가 피드백을 써주는 웹 서버
- ③ 쇼핑몰 결제 시스템
- ④ 사진 편집 앱

✅ [답지와 해설]

1. ② — 요청은 항상 main(교통정리)→router(처리)→database(연결)→models(DB)→schemas(모양)→응답 순. **2.** ② — main.py는 안내데스크. 앱을 켜고 요청을 라우터로 연결. **3.** ③ — router는 주방 코너. 실제 넣기/꺼내기 로직 처리. **4.** ③ — models.py = 창고 선반 설계도(DB 테이블의 진짜 모양). **5.** ② — schemas.py = 주문서/영수증 양식. 데이터 검사. **6.** ② — database.py는 문지기. get_db로 연결 열고 닫음. **7.** ② — 역할별로 나눠야 관리가 쉬워서. 한 파일에 다 넣으면 괴물이 됨. **8.** ② — models→schemas→router→main. 새 기능마다 반복되는 뼈대. **9.** ③ — Day 2 대상은 아이(Child). **10.** ② — Day 3 대상은 수업기록(LessonRecord). **11.** ③ — Day 5 대상은 피드백(Feedback). **12.** ② — POST=새 데이터 생성, PATCH=기존 데이터 일부 수정. **13.** ② — exclude_unset=True는 사용자가 실제 보낸 필드만. None 덮어쓰기 방지. **14.** ② — setattr(객체,"속성",값) = 객체.속성 = 값. **15.** ① — 한쪽만 있으면 서버 켤 때 에러. 양쪽에 다 써야 함. **16.** ③ — 전체는 `.all()`, 하나는 `.first()`. **17.** ② — 왼쪽=DB 컬럼(저장된 것), 오른쪽=URL로 들어온 값. **18.** ② — id는 그 표에서 자기 자신의 고유 번호(기본키). **19.** ① — child_id는 children의 id를 베껴 적은 연결고리(외래키). **20.** ② — add(담기)→commit(저장확정)→refresh(최신상태 다시 읽기). **21.** ② — server_default=func.now()로 DB가 자동 입력해서 안 보냄. **22.** ② — Pydantic v2에서 orm_mode → from_attributes. **23.** ② — 연습 데이터라 drop_all로 싹 지우고 create_all로 재생성. (실무는 Alembic) **24.** ② — prompt_service는 글 조립만. AI는 아직 안 부름(주문서 작성). **25.** ② — llm_service는 조립된 글을 Gemini에 보내고 답 받기(주방에 주문). **26.** ① — f-string: 문자열 앞 f + {변수} 자리에 값이 들어감. **27.** ② — 함수 밖에 두면 프로그램 켤 때 1번만. 매번 다시 하는 낭비 방지. **28.** ③ — .env에 숨기고 .gitignore로 제외. 코드/깃허브에 노출 금지. **29.** ② — json.loads: 문자열 → 파이썬 딕셔너리. 그래야 data["key"]로 꺼냄. **30.** ② — 없는 ID로 500 크래시 전에 404 "없음"을 깔끔히 알림. **31.** ② — 리스트 응답은 List[...]로 감쌈. `from typing import List` 필요. **32.** ② — from_attributes로 DB 객체를 Pydantic이 읽어 JSON 변환 허락. **33.** ② — 매개변수 = 부를 때 채워질 빈 그릇. 호출하는 쪽이 값을 넣음. **34.** ② — 위쪽 "Server response"의 Code가 진짜 결과. 아래 표는 문서용. **35.** ② — import + include_router 2줄로 연결해야 작동. **36.** ② — main→router→database→models→schemas→응답. **37.** ② — .all()=여러 개(리스트), .first()=딱 하나. **38.** ② — Create=받을 때, Response=줄 때(보통 id·created_at 추가). **39.** ③ — rag_context는 Day 8(RAG)에서 채우기로 예약된 빈칸. **40.** ② — f-string 안 진짜 중괄호는 `{{, }}` 두 개씩. (JSON 예시 넣을 때 필요) **41.** ② — \n = 줄바꿈(엔터). 없으면 한 줄에 다 붙음. **42.** ② — += 는 기존 문자열 뒤에 더 붙이기(누적). **43.** ② — 파라미터가 클래스 이름과 같으면 Python이 헷갈림. record 같은 다른 이름 사용. **44.** ② — key 철자가 딕셔너리와 안 맞으면 KeyError. 프롬프트 지정 이름 그대로. **45.** ② — 이 프로젝트는 Google Gemini(AI Studio) 사용. **46.** ② — 인증됐는데 429면 코드가 아니라 할당량/모델 문제. 모델 바꾸면 해결되기도. **47.** ② — 돌리기 전 venv 켜졌나 `(kids-feedback)` 표시 확인. **48.** ② — Swagger는 자동 생성 API 테스트 화면(개발자용). 진짜 화면은 Day 12~14 Streamlit. **49.** ② — 완료 기준은 노트 양이 아니라 내 말로 30초 설명 가능한지. **50.** ② — 강사가 넣고(POST)·꺼내보고(GET)·AI가 피드백 써주는 웹 서버.